

EXPERIMENT 1

IMPLEMENTATION OF CAESAR CIPHER ENCRYPTION AND DECRYPTION

AIM

To implement the Caesar Cipher algorithm for the encryption and decryption of plaintext using a fixed shift key.

ALGORITHM

Encryption

Step 1: Start the program.

Step 2: Read the plaintext from the user.

Step 3: Read the shift key value.

Step 4: Traverse each character of the plaintext.

Step 5: If the character is an alphabet, encrypt it by shifting it forward using the key value with modulo 26.

Step 6: If the character is a space or special character, keep it unchanged.

Step 7: Store the encrypted characters to form the ciphertext.

Step 8: Display the encrypted text.

Decryption

Step 9: Traverse each character of the ciphertext.

Step 10: Use the same shift key for decryption.

Step 11: If the character is an alphabet, decrypt it by shifting it backward using the key value with modulo 26.

Step 12: If the character is a space or special character, keep it unchanged.

Step 13: Store the decrypted characters to reconstruct the original plaintext.

Step 14: Display the decrypted text.

Step 15: Stop the program

CODE:

```
main.c
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main()
6  {
7      char text[100], cipher[100], decrypt[100];
8      int key, i;
9
10     printf("Enter Plain Text: ");
11     fgets(text, sizeof(text), stdin);
12
13     printf("Enter Shift Key: ");
14     scanf("%d", &key);
15
16     // Encryption
17     for(i = 0; text[i] != '\0'; i++)
18     {
19         if(isupper(text[i]))
20             cipher[i] = ((text[i] - 'A' + key) % 26) + 'A';
21
22         else if(islower(text[i]))
23             cipher[i] = ((text[i] - 'a' + key) % 26) + 'a';
24
25         else
26             cipher[i] = text[i];
27     }
28     cipher[i] = '\0';
29
30     // Decryption
31     for(i = 0; cipher[i] != '\0'; i++)
32     {
33         if(isupper(cipher[i]))
34             decrypt[i] = ((cipher[i] - 'A' - key + 26) % 26) + 'A';
35
36         else if(islower(cipher[i]))
37             decrypt[i] = ((cipher[i] - 'a' - key + 26) % 26) + 'a';
38
39         else
40             decrypt[i] = cipher[i];
41     }
42     decrypt[i] = '\0';
43
44     printf("\nAlgorithm : Caesar Cipher");
45     printf("\nEncrypted Text : %s", cipher);
46     printf("\nDecrypted Text : %s", decrypt);
47
48     return 0;
49 }
```

OUTPUT:

```
Output
Enter Plain Text: hello world
Enter Shift Key: 3

Algorithm : Caesar Cipher
Encrypted Text : khoor zruog

Decrypted Text : hello world

=== Code Execution Successful ===
```

RESULT

Thus, the Caesar Cipher encryption and decryption algorithm was successfully implemented using the C programming language. The plaintext was encrypted using the specified shift key, and the original plaintext was successfully recovered through the decryption process

EXPERIMENT 2

IMPLEMENTATION OF MONOALPHABETIC SUBSTITUTION CIPHER ENCRYPTION AND DECRYPTION

AIM

To implement the Monoalphabetic Substitution Cipher algorithm for encryption and decryption of plaintext using a substitution key.

ALGORITHM

Encryption

Step 1: Start the program.

Step 2: Read the plaintext from the user.

Step 3: Define the substitution key.

Step 4: Traverse each character of the plaintext.

Step 5: Replace each alphabet with its corresponding character in the substitution key.

Step 6: Keep spaces and special characters unchanged.

Step 7: Display the encrypted text.

Decryption

Step 8: Traverse each character of the encrypted text.

Step 9: Find the corresponding character in the original alphabet.

Step 10: Keep spaces and special characters unchanged.

Step 11: Display the decrypted text.

Step 12: Stop the program

CODE

```
1 # Algorithm: Monoalphabetic Substitution Cipher Encryption and Decryption
2
3 def encrypt(text, key):
4     alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
5     cipher = ""
6
7     for ch in text.upper():
8         if ch in alphabet:
9             cipher += key[alphabet.index(ch)]
10        else:
11            cipher += ch
12
13    return cipher
14
15
16 def decrypt(cipher, key):
17     alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
18     text = ""
19
20     for ch in cipher.upper():
21         if ch in key:
22             text += alphabet[key.index(ch)]
23        else:
24            text += ch
25
26    return text
27
28
29 print("Algorithm: Monoalphabetic Substitution Cipher Encryption and Decryption")
30
31 text = input("Enter text: ")
32 key = input("Enter 26 letter key: ").upper()
33
34 encrypted = encrypt(text, key)
35 print("Encrypted:", encrypted)
36
37 decrypted = decrypt(encrypted, key)
38 print("Decrypted:", decrypted)
```

OUTPUT

Output
Algorithm: Monoalphabetic Substitution Cipher Encryption and Decryption
Enter text: chethrika
Enter 26 letter key: QWERTYUIOPASDFGHJKLZXCVBNM
Encrypted: EITZIKOAQ
Decrypted: CHETHRIKA
=== Code Execution Successful ===

RESULT

Thus, the Monoalphabetic Substitution Cipher encryption and decryption algorithm was successfully implemented using the C programming language. The plaintext was encrypted using the substitution key and successfully decrypted to obtain the original plaintext

EXPERIMENT 3

IMPLEMENTATION OF POLYALPHABETIC (VIGENÈRE) CIPHER ENCRYPTION AND DECRYPTION

AIM

To implement the Polyalphabetic (Vigenère) Cipher algorithm for encrypting and decrypting a message using a keyword.

ALGORITHM

Encryption

Step 1: Start the program.

Step 2: Read the plaintext from the user.

Step 3: Read the keyword.

Step 4: Repeat the keyword until it matches the length of the plaintext.

Step 5: Traverse each character of the plaintext.

Step 6: If the character is an alphabet, encrypt it using the corresponding character of the repeated keyword.

Step 7: If the character is a space or special character, keep it unchanged.

Step 8: Store the encrypted characters to form the ciphertext.

Step 9: Display the encrypted text.

Decryption

Step 10: Traverse each character of the ciphertext.

Step 11: Use the same repeated keyword for decryption.

Step 12: If the character is an alphabet, decrypt it using the corresponding keyword character.

Step 13: If the character is a space or special character, keep it unchanged.

Step 14: Store the decrypted characters to reconstruct the original plaintext.

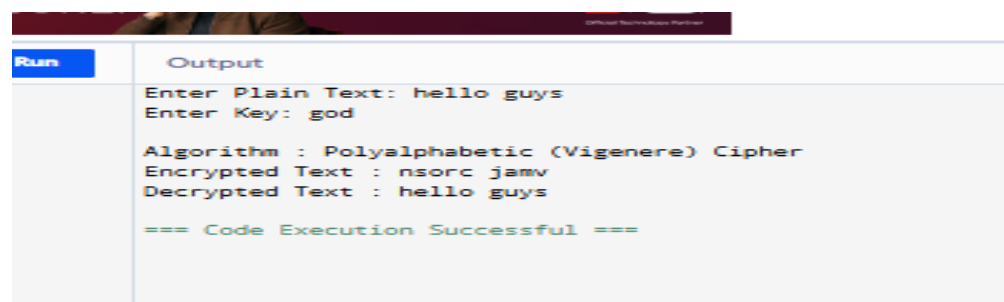
Step 15: Display the decrypted text.

Step 16: Stop the program.

CODE

```
main.c
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main()
6  {
7      char p[100], k[20], c[100], d[100];
8      int i, j = 0, s;
9
10     printf("Enter Plain Text: ");
11     fgets(p, 100, stdin);
12     p[strcspn(p, "\n")] = 0;
13
14     printf("Enter Key: ");
15     scanf("%s", k);
16
17     // Encryption
18     for(i = 0; p[i]; i++)
19         if(isalpha(p[i]))
20         {
21             s = toupper(k[j++ % strlen(k)]) - 'A';
22             c[i] = isupper(p[i]) ? ((p[i] - 'A' + s) % 26) + 'A' : ((p[i] - 'a' + s) % 26) + 'a';
23         }
24         else
25             c[i] = p[i];
26     c[i] = '\0';
27
28     // Decryption
29     j = 0;
30     for(i = 0; c[i]; i++)
31         if(isalpha(c[i]))
32         {
33             s = toupper(k[j++ % strlen(k)]) - 'A';
34             d[i] = isupper(c[i]) ? ((c[i] - 'A' - s + 26) % 26) + 'A' : ((c[i] - 'a' - s + 26) % 26) + 'a';
35         }
36         else
37             d[i] = c[i];
38     d[i] = '\0';
39
40     printf("\nAlgorithm : Polyalphabetic (Vigenere) Cipher");
41     printf("\nEncrypted Text : %s", c);
42     printf("\nDecrypted Text : %s", d);
43
44     return 0;
45 }
```

OUTPUT



```
Run      Output
Enter Plain Text: hello guys
Enter Key: god

Algorithm : Polyalphabetic (Vigenere) Cipher
Encrypted Text : nsorc jamv
Decrypted Text : hello guys

=== Code Execution Successful ===
```

RESULT

Thus, the Polyalphabetic (Vigenère) Cipher encryption and decryption algorithm was successfully implemented using the C programming language. The plaintext was encrypted using the given keyword, and the original plaintext was successfully recovered through the decryption process. Hence, the correctness of the algorithm was verified

EXPERIMENT 4

IMPLEMENTATION OF HILL CIPHER ENCRYPTION AND DECRYPTION

AIM

To implement the Hill Cipher algorithm for encrypting and decrypting a message using a 2×2 key matrix.

ALGORITHM

Encryption

Step 1: Start the program.

Step 2: Read the plaintext from the user.

Step 3: Read the 2×2 key matrix.

Step 4: Convert each plaintext character into its numerical equivalent ($A = 0, B = 1, \dots, Z = 25$).

Step 5: Divide the plaintext into blocks of two characters.

Step 6: Multiply each plaintext block by the key matrix and apply modulo 26.

Step 7: Convert the resulting numerical values back into alphabets.

Step 8: Display the encrypted text.

Decryption

Step 9: Read the inverse of the key matrix modulo 26.

Step 10: Convert the ciphertext into numerical values.

Step 11: Multiply each ciphertext block by the inverse key matrix and apply modulo 26.

Step 12: Convert the resulting numerical values back into alphabets.

Step 13: Display the decrypted text.

Step 14: Stop the program.

CODE

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char text[100];
7      int key[2][2], inv[2][2];
8      int i, a, b, c1, c2, p1, p2;
9
10     printf("Enter Plain Text (CAPITAL LETTERS): ");
11     scanf("%s", text);
12
13     printf("Enter 2x2 Key Matrix:\n");
14     for(i=0; i<2; i++)
15         scanf("%d %d", &key[i][0], &key[i][1]);
16
17     printf("Enter 2x2 Inverse Key Matrix:\n");
18     for(i=0; i<2; i++)
19         scanf("%d %d", &inv[i][0], &inv[i][1]);
20
21     printf("\nAlgorithm : Hill Cipher");
22
23     printf("\nEncrypted Text : ");
24
25     for(i=0; text[i]!='\0'; i+=2)
26     {
27         a=text[i]-'A';
28
29         if(text[i+1]!='\0')
30             b=text[i+1]-'A';
31         else
32             b='X'-'A';
33
34         c1=(key[0][0]*a + key[0][1]*b)%26;
35         c2=(key[1][0]*a + key[1][1]*b)%26;
36
37         text[i]=c1+'A';
38         text[i+1]=c2+'A';
39
40         printf("%c%c", text[i], text[i+1]);
41     }
42
43     printf("\nDecrypted Text : ");
44
45     for(i=0; text[i]!='\0'; i+=2)
46     {
47         a=text[i]-'A';
48         b=text[i+1]-'A';
49
50         p1=(inv[0][0]*a + inv[0][1]*b)%26;
51         p2=(inv[1][0]*a + inv[1][1]*b)%26;
52
53         printf("%c%c", p1+'A', p2+'A');
54     }
55 }
```

OUTPUT

```
Output
Enter Plain Text (CAPITAL LETTERS): HELLO
Enter 2x2 Key Matrix:
2 3
1 2
Enter 2x2 Inverse Key Matrix:
2 1
3 2

Algorithm : Hill Cipher
Encrypted Text : APDHTI
Decrypted Text : PENXUV

=== Code Execution Successful ===
```

RESULT:

Thus, the Hill Cipher algorithm was successfully implemented for both encryption and decryption, and the results were verified successfully..

EXPERIMENT 5

IMPLEMENTATION OF HILL CIPHER ENCRYPTION AND DECRYPTION

AIM

To implement the Playfair Cipher algorithm for encryption and decryption of a message using a keyword.

ALGORITHM

Step 1: Start.

Step 2: Read the keyword and plaintext.

Step 3: Create a 5×5 key matrix using the keyword (merge I/J).

Step 4: Convert the plaintext to uppercase and remove spaces.

Step 5: Replace J with I.

Step 6: Divide the plaintext into pairs.

Step 7: Insert **X** between repeated letters and add **X** if the last pair is incomplete.

Step 8: Encrypt each pair:

- Same row → Replace with the letter to the right.
- Same column → Replace with the letter below.
- Different row and column → Apply the rectangle rule.

Step 9: Combine all encrypted pairs to get the ciphertext.

Step 10: Decrypt using the reverse rules:

- Same row → Letter to the left.
- Same column → Letter above.
- Rectangle → Apply the rectangle rule.

Step 11: Display the encrypted and decrypted text.

Step 12: Stop.

CODE

```
1- def key_matrix(key):
2     key = key.upper().replace("J", "I")
3     s = ""
4     for ch in key + "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
5         if ch.isalpha() and ch not in s:
6             s += ch
7     return [list(s[i:i+5]) for i in range(0, 25, 5)]
8
9- def find(mat, ch):
0     if ch == "J":
1         ch = "I"
2     for i in range(5):
3         for j in range(5):
4             if mat[i][j] == ch:
5                 return i, j
6
7- def prepare(text):
8     text = text.upper().replace(" ", "").replace("J", "I")
9     p = ""
0     i = 0
1     while i < len(text):
2         a = text[i]
3         if i + 1 < len(text):
4             b = text[i + 1]
5             if a == b:
6                 p += a + "X"
7                 i += 1
8             else:
9                 p += a + b
0                 i += 2
1         else:
2             p += a + "X"
3             i += 1
4     return p
5
6- def encrypt(mat, text):
7     text = prepare(text)
```

```
41         r2, c2 = find(mat, text[i+1])
42
43         if r1 == r2:
44             c += mat[r1][(c1+1)%5] + mat[r2][(c2+1)%5]
45         elif c1 == c2:
46             c += mat[(r1+1)%5][c1] + mat[(r2+1)%5][c2]
47         else:
48             c += mat[r1][c2] + mat[r2][c1]
49     return c
50
51- def decrypt(mat, text):
52     p = ""
53     for i in range(0, len(text), 2):
54         r1, c1 = find(mat, text[i])
55         r2, c2 = find(mat, text[i+1])
56
57         if r1 == r2:
58             p += mat[r1][(c1-1)%5] + mat[r2][(c2-1)%5]
59         elif c1 == c2:
60             p += mat[(r1-1)%5][c1] + mat[(r2-1)%5][c2]
61         else:
62             p += mat[r1][c2] + mat[r2][c1]
63     return p
64
65     key = input("Enter Key: ")
66     plain = input("Enter Plaintext: ")
67
68     mat = key_matrix(key)
69
70     print("\nKey Matrix:")
71     for row in mat:
72         print(*row)
73
74     cipher = encrypt(mat, plain)
75     print("Encrypted Text:", cipher)
76
77     plain = decrypt(mat, cipher)
78     print("Decrypted Text:", plain)
```

OUTPUT

```
Output
Enter Key: monarchy
Enter Plaintext: happy

Key Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z
Encrypted Text: BOSVQH
Decrypted Text: HAPXPY

=== Code Execution Successful ===
```

RESULT

Thus, the Playfair Cipher algorithm was successfully implemented, and the given plaintext was encrypted and decrypted using the specified keyword.

EXPERIMENT 6

IMPLEMENTATION OF RAIL FENCE CIPHER ENCRYPTION AND DECRYPTION

AIM:

To implement Rail Fence Cipher algorithm for encryption and decryption of a given message.

ALGORITHM:

Encryption:

Step 1: Start the program.

Step 2: Read the plaintext and number of rails.

Step 3: Place the plaintext characters in a zigzag pattern across the rails.

Step 4: Read the characters row by row from the rails.

Step 5: Generate and display the ciphertext.

Decryption:

Step 1: Start the program.

Step 2: Read the ciphertext and number of rails.

Step 3: Create an empty zigzag pattern with the given rails.

Step 4: Mark the positions of characters in the zigzag pattern.

Step 5: Fill the marked positions with ciphertext characters row by row.

Step 6: Read the characters in zigzag order to obtain the plaintext.

Step 7: Display the decrypted message.

Step 8: Stop the program.

CODE

```
1- def encrypt(text, rails):
2     fence = [''] * rails
3     row, step = 0, 1
4
5     for ch in text:
6         fence[row] += ch
7         if row == 0:
8             step = 1
9         elif row == rails-1:
10            step = -1
11        row += step
12
13    return ''.join(fence)
14
15 def decrypt(cipher, rails):
16     pattern = [[''] * len(cipher) for _ in range(rails)]
17     row, step = 0, 1
18
19     for i in range(len(cipher)):
20         pattern[row][i] = '*'
21         if row == 0:
22             step = 1
23         elif row == rails-1:
24             step = -1
25         row += step
26
27     index = 0
28     for r in range(rails):
29         for c in range(len(cipher)):
30             if pattern[r][c] == '*':
31                 pattern[r][c] = cipher[index]
32                 index += 1
33
34     result = ''
35     row, step = 0, 1
36     for i in range(len(cipher)):
37         result += pattern[row][i]
```

```
38         result += pattern[row][i]
39         if row == 0:
40             step = 1
41         elif row == rails-1:
42             step = -1
43         row += step
44
45     return result
46
47 text = input("Enter text: ")
48 rails = int(input("Enter number of rails: "))
49
50 cipher = encrypt(text, rails)
51 print("Encrypted:", cipher)
52
53 print("Decrypted:", decrypt(cipher, rails))
```

OUTPUT

```
Output
Enter text: hello world
Enter number of rails: 3
Encrypted: horel ollwd
Decrypted: hello world

=== Code Execution Successful ===
```

RESULT:

Thus, the Rail Fence Cipher encryption and decryption was implemented successfully.

EXPERIMENT 7

IMPLEMENTATION OF COLUMNAR TRANSPOSITION CIPHER ENCRYPTION AND DECRYPTION

AIM:

To implement Columnar Transposition Cipher algorithm for encryption and decryption of a given message.

ALGORITHM:

Encryption:

Step 1: Start the program.

Step 2: Read the plaintext and key from the user.

Step 3: Arrange the plaintext characters into rows according to the key length.

Step 4: Arrange the columns based on the alphabetical order of the key.

Step 5: Read the characters column by column to generate the ciphertext.

Step 6: Display the encrypted message.

Decryption:

Step 1: Start the program.

Step 2: Read the ciphertext and key from the user.

Step 3: Calculate the number of rows and columns using the key length.

Step 4: Arrange the ciphertext characters into columns according to the key order.

Step 5: Read the characters row by row to obtain the plaintext.

Step 6: Display the decrypted message.

Step 7: Stop the program.

CODE

```
1- def encrypt(text, key):
2     text = text.replace(" ", "")
3     col = len(key)
4
5     table = []
6-     for i in range(0, len(text), col):
7         table.append(list(text[i:i+col]))
8
9-     while len(table[-1]) < col:
10        table[-1].append('X')
11
12    order = sorted(range(col), key=lambda x: key[x])
13
14    cipher = ""
15-    for i in order:
16-        for row in table:
17            cipher += row[i]
18
19    return cipher
20
21
22- def decrypt(cipher, key):
23    col = len(key)
24    rows = len(cipher) // col
25
26    order = sorted(range(col), key=lambda x: key[x])
27
28    table = [[''] * col for _ in range(rows)]
29
30    index = 0
31-    for i in order:
32-        for j in range(rows):
33            table[j][i] = cipher[index]
34            index += 1
35
36    text = ""
37-    for row in table:
38        text += ''.join(row)
```

```
38        text += ''.join(row)
39
40    return text.replace("X", "")
41
42
43 text = input("Enter text: ")
44 key = input("Enter key: ")
45
46 cipher = encrypt(text, key)
47
48 print("Encrypted:", cipher)
49 print("Decrypted:", decrypt(cipher, key))
```


OUTPUT

```
Output
Enter text: HELLO WORLD
Enter key: ZEBRA
Encrypted: ODLREOLLHW
Decrypted: HELLOWORLD

=== Code Execution Successful ===
```

RESULT:

Thus, the Columnar Transposition Cipher encryption and decryption was implemented successfully. The given plaintext was converted into ciphertext using columnar technique. The original plaintext was obtained successfully after decryption.

